



Advanced Digital Design

PRESENTED BY:

Dr. Shasanka Sekhar Rout

Session 10 & 11

Lecture: Counter Design and Frequency Division

- ✓ Counters: Introduction and types
- ✓ Designing of Counters using FSM approach: MOD-N, Binary, Decade
- ✓ Binary Counter as a Frequency divider
- ✓ Frequency division hazards

Counter Design

Counters:

- A **counter** is a **sequential circuit** that counts clock pulses. It is a type of **register** that goes through a sequence of states based on the clock input.
- It is used in **timers, event counting, frequency division, and digital clocks.**
- A counter that follows the binary number sequence is called a *binary counter*.
- An n -bit binary counter consists of n flip-flops and can count in binary from 0 through $2^n - 1$.

Types of Counters:

1. Asynchronous (Ripple) Counter

- ✓ Each flip-flop's output acts as the clock for the next flip-flop
- ✓ Slower due to propagation delay
- ✓ Simple design, used for low-speed applications
- ◆ Example: 4-bit Ripple Counter (Counts 0 to 15, then resets)

Counter Design

2. Synchronous Counter

- ✓ All flip-flops are triggered by the same clock pulse
- ✓ Faster operation than ripple counters
- ✓ No propagation delay issues
- ◆ Example: 4-bit Synchronous Counter (Counts 0 to 15 in binary order)

3. Up Counter and Down Counter

- ✓ **Up Counter:** Increments count ($0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \dots$)
- ✓ **Down Counter:** Decrements count ($15 \rightarrow 14 \rightarrow 13 \dots \rightarrow 0$)
- ✓ **Up-Down Counter:** Can count in both directions

4. Modulus (MOD) Counters

- ✓ MOD-n counter counts up to 'n' states before resetting to 0
- ✓ Example: MOD-10 counter (Decade Counter) counts 0-9
- ◆ **Formula:**

For a MOD-n counter, the number of flip-flops required is:

$$N = \log_2(n)$$

where N is the number of flip-flops.

Counter Design

5. Ring Counter

- ✓ A shift register with feedback
- ✓ Only one flip-flop is set at a time
- ✓ Requires a preset value
- ◆ Example: 4-bit Ring Counter (Counts 4 unique states instead of 16)

6. Johnson Counter (Twisted Ring Counter)

- ✓ Modified ring counter where inverted output is fed back
- ✓ Counts $2n$ states (n = number of flip-flops)
- ◆ Example: 4-bit Johnson Counter (Counts 8 unique states instead of 16)

Counter Type	Clocking Mechanism	Speed	Usage
Asynchronous (Ripple)	Each flip-flop gets input from the previous	Slow (Propagation delay)	Low-speed applications
Synchronous	All flip-flops triggered by the same clock	Fast	High-speed digital systems
Up/Down	Counts in increasing or decreasing order	Moderate	Frequency division, bidirectional counters
Ring	Feedback mechanism	Moderate	Shift registers, pattern generation
Johnson	Modified ring counter	Fast	Light chasers, sequence detectors

Counter Design

State & Modulus of Counter:

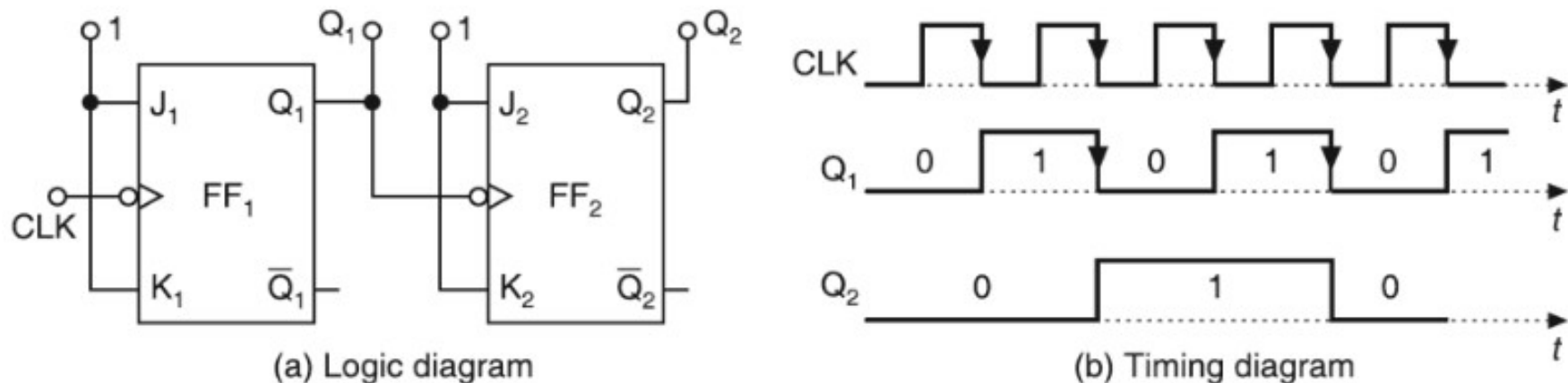
- ✓ Each of the counts of the counter is called the *state* of the counter.
- ✓ The number of states through which the counter passes before returning to the starting state is called the *modulus* of the counter.
- ✓ Since a 2-bit counter has 4 states; it is called a mod-4 counter. It divides the input clock signal frequency by 4, therefore, it is also called a divide-by-4 counter. It requires 2 FFs.
- ✓ In general, an n -bit counter will have n FFs and 2^n states, and divides the input frequency by 2^n . Hence, it is a *divide-by- 2^n counter*.
- ✓ A counter which goes through all the possible states before restarting is called the *full modulus counter*.
- ✓ A counter in which maximum number of states can be changed is called the *variable modulus counter*.
- ✓ The final state of the counter sequence is called the *terminal count*.

Counter Design

Designing of Counters using FSM approach: MOD-N, Binary, Decade):

2-bit Ripple Up-counter using Negative Edge-triggered Flip-Flops:

- The 2-bit up-counter counts in the order 0, 1, 2, 3, 0, 1, ..., i.e. 00, 01, 10, 11, 00, 01, ..., etc.
- Figure shows a 2-bit ripple up-counter, using negative edge-triggered J-K FFs, and its timing diagram.

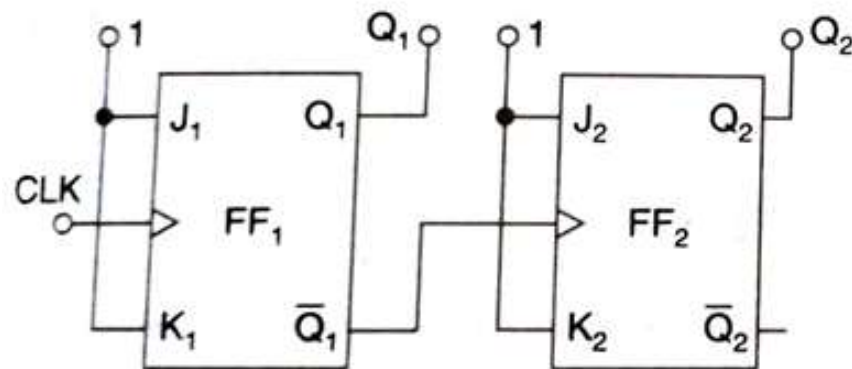


Asynchronous 2-bit up-counter using negative edge-triggered flip-flops.

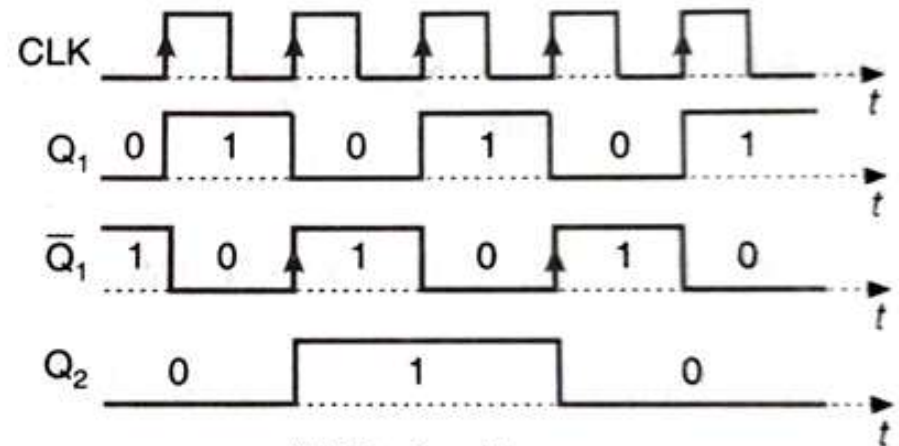
Counter Design

2-bit Ripple Up-counter using Positive Edge-triggered Flip-Flops:

- A 2-bit ripple up-counter, using positive edge-triggered J-K FFs, and its timing diagram are shown in Figure.
- The counting sequence is 00, 01, 10, 11, 00, 01, ..., etc.



(a) Logic diagram



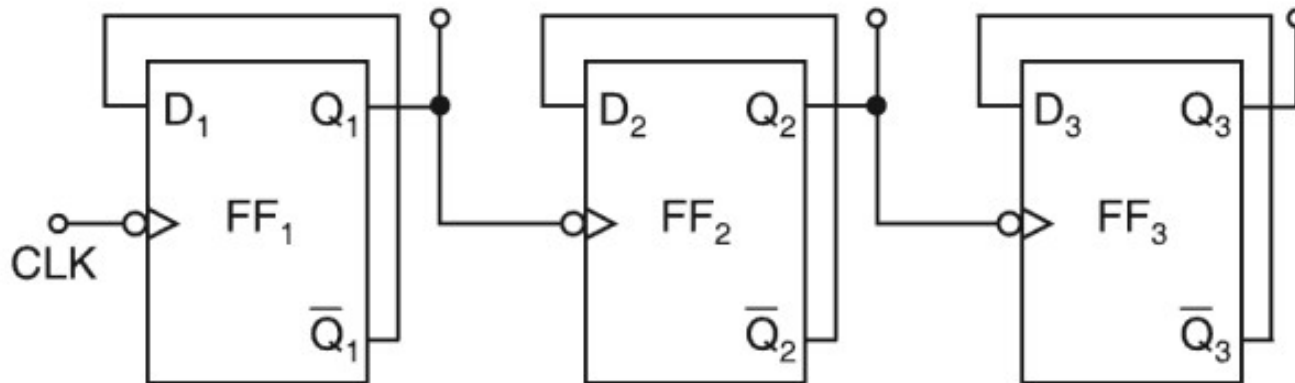
(b) Timing diagram

Asynchronous 2-bit up-counter using positive-edge triggered J-K flip-flops.

Counter Design

3-bit Ripple Counter using D FFs:

- For ripple counters, the FFs used must be in toggle mode. The D FFs may be used in toggle mode by connecting the $\bar{Q}$ of each FF to its D terminal.
- The 3-bit ripple counter using D FFs is shown in Figure.

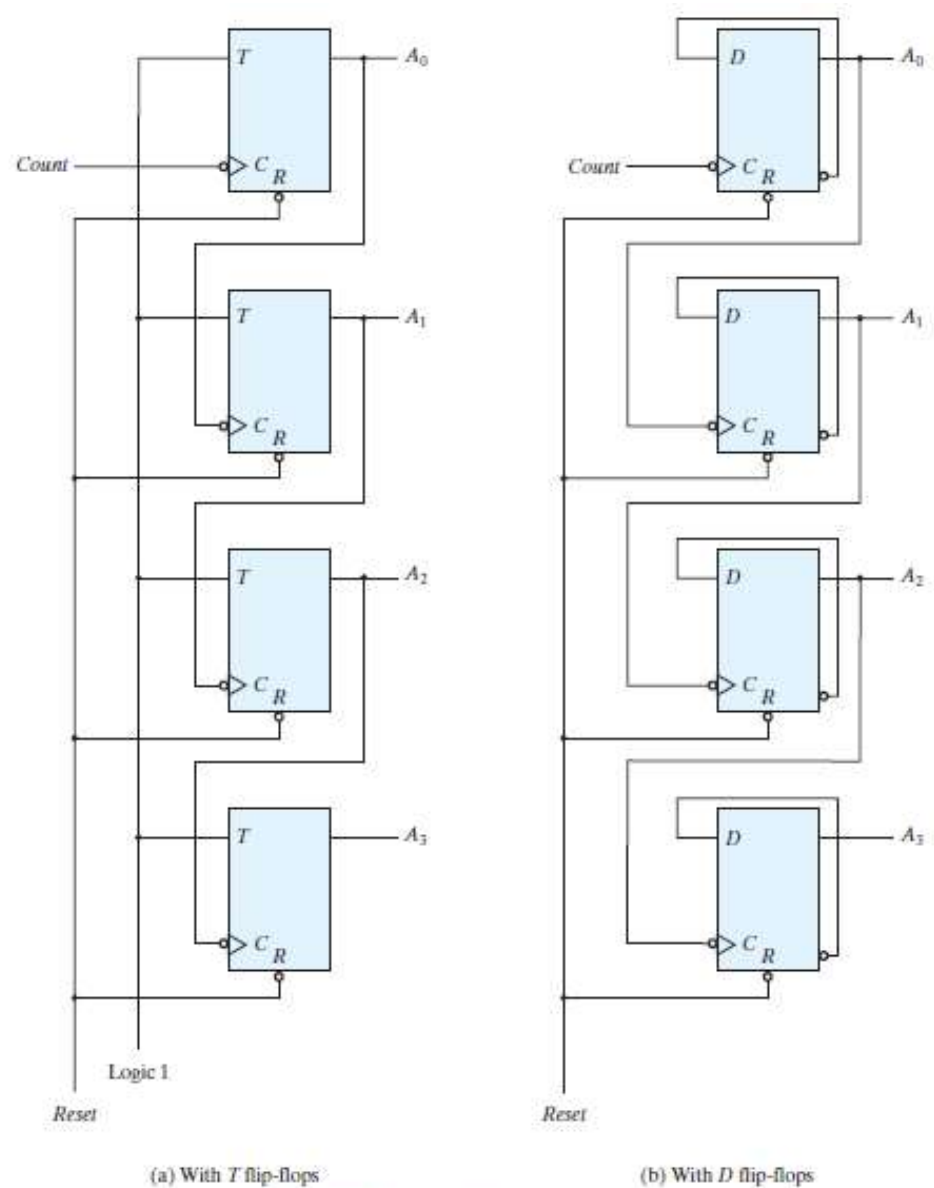


Logic diagram of a 3-bit asynchronous counter using D flip-flops.

Counter Design

4-bit Binary Ripple Counters:

- The logic diagram of two 4-bit binary ripple counters is shown in Figure.
- The counter is constructed with complementing flip-flops of the T type in part (a) and D type in part (b).
- The output of each flip-flop is connected to the C input of the next flip-flop in sequence.
- The flip-flop holding the least significant bit receives the incoming count pulses.

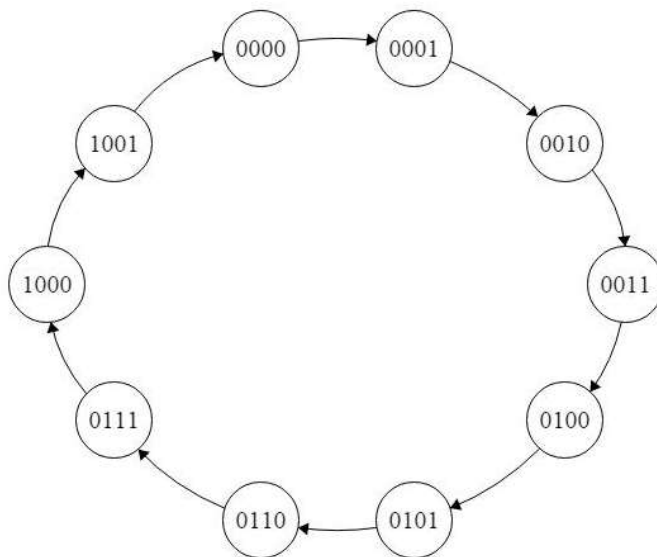


Four-bit binary ripple counter

Counter Design

Design of a Mod-10 Asynchronous Counter using T FFs:

- A mod-10 counter is a **decade counter**. It is also called a **BCD counter** or a **divide-by-10 counter**.
- It requires four FFs (the smallest value of n satisfying the condition $10 \leq 2^n$, is $n = 4$). So, there are 16 possible states, out of which ten are valid and the remaining six are invalid.
- The counter has ten stable states, 0000 through 1001, i.e. it counts from 0 to 9.
- The state 1010 is a temporary state for which the reset signal $R = 1$, $R = 0$ for 0000 to 1001, and $R = X$ (don't care) for 1011 to 1111.



(a) State digram

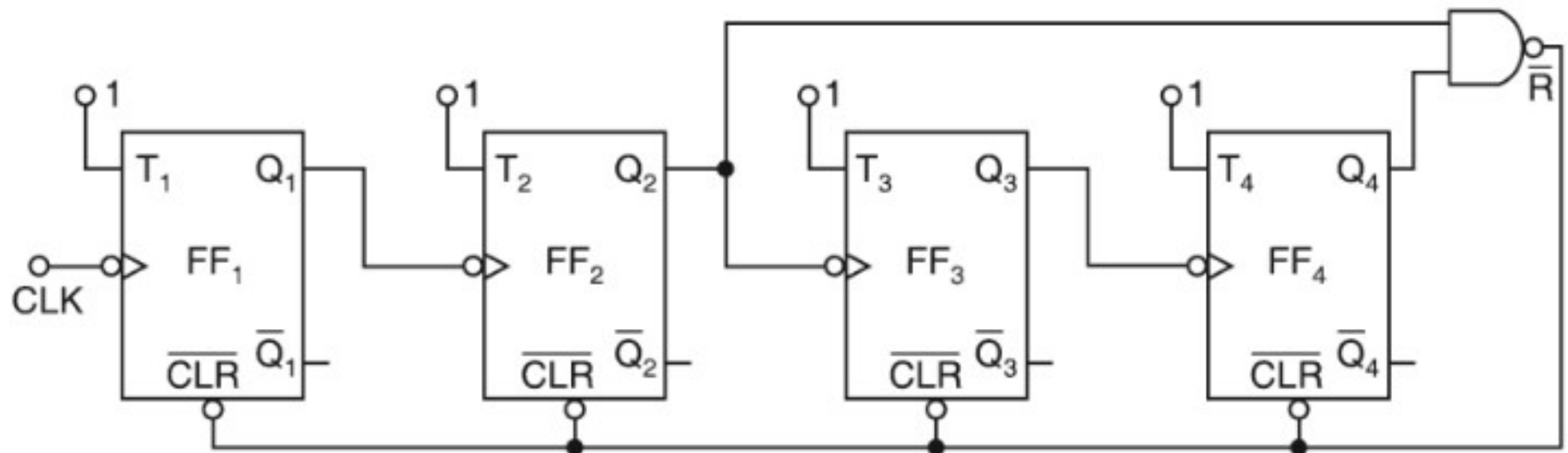
Q_2Q_1					
Q_4Q_3		00	01	11	10
	00	0	1	3	2
	01	4	5	7	6
	11	12 x	13 x	15 x	14 x
	10	8	9	11 x	10 1

$$R = Q_4Q_2$$

(b) K-Map

Counter Design

Design of a Mod-10 Asynchronous Counter using T FFs:



(c) Logic diagram

Asynchronous mod-10 counter using T flip-flops.

Counter Design

Design of Synchronous Counters:

- A common clock triggers all flip-flops simultaneously, rather than one at a time in succession as in a ripple counter.
- *Step 1. Number of flip-flops:* Based on the description of the problem, determine the required number n of the FFs – the smallest value of n is such that the number of states $N \leq 2^n$ – and the desired counting sequence.
- *Step 2. State diagram:* Draw the state diagram showing all the possible states.
- *Step 3. Choice of flip-flops and excitation table:* Select the type of flip-flops to be used and write the excitation table that lists the present state (PS), the next state (NS) and the required excitation.
- *Step 4. Minimal expressions for the excitations:* Obtain the minimal expressions for the excitations of the FFs using K-maps drawn for the excitations of the flip-flops in terms of present state and inputs.
- *Step 5. Logic diagram:* Draw a logic diagram based on the minimal expressions.

Counter Design

Excitation tables

PS	NS	Required inputs	
Q_n	Q_{n+1}	S	R
0	0	0	×
0	1	1	0
1	0	0	1
1	1	×	0

(a) S-R FF excitation table

PS	NS	Required inputs	
Q_n	Q_{n+1}	J	K
0	0	0	×
0	1	1	×
1	0	×	1
1	1	×	0

(b) J-K FF excitation table

PS	NS	Required input
Q_n	Q_{n+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

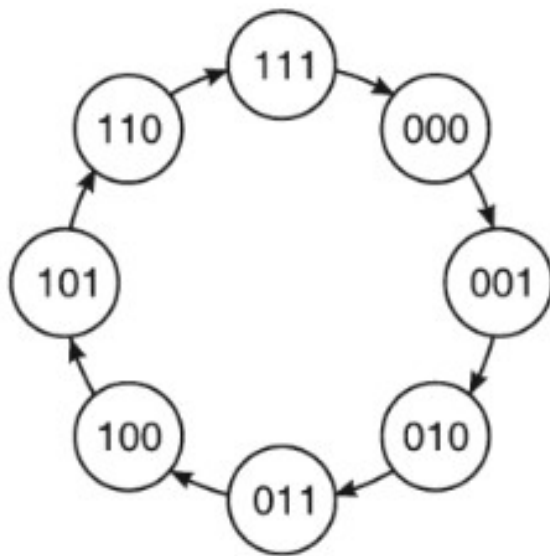
(c) D FF excitation table

PS	NS	Required input
Q_n	Q_{n+1}	T
0	0	0
0	1	1
1	0	1
1	1	0

(d) T FF excitation table

Counter Design

Design of a Synchronous 3-bit Up Counter using J-K FFs:



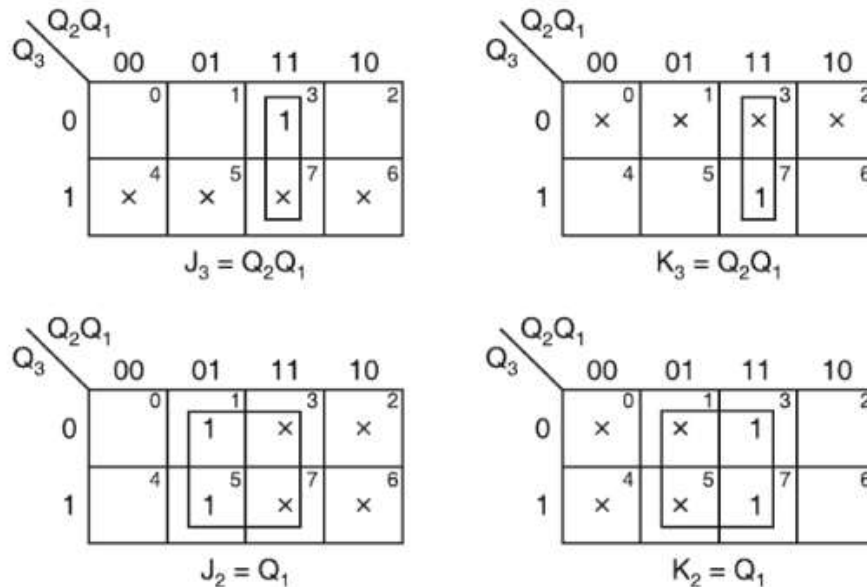
(a) State diagram

PS			NS			Required excitations					
Q_3	Q_2	Q_1	Q_3	Q_2	Q_1	J_3	K_3	J_2	K_2	J_1	K_1
0	0	0	0	0	1	0	x	0	x	1	x
0	0	1	0	1	0	0	x	1	x	x	1
0	1	0	0	1	1	0	x	x	0	1	x
0	1	1	1	0	0	1	x	x	1	x	1
1	0	0	1	0	1	x	0	0	x	1	x
1	0	1	1	1	0	x	0	1	x	x	1
1	1	0	1	1	1	x	0	x	0	1	x
1	1	1	0	0	0	x	1	x	1	x	1

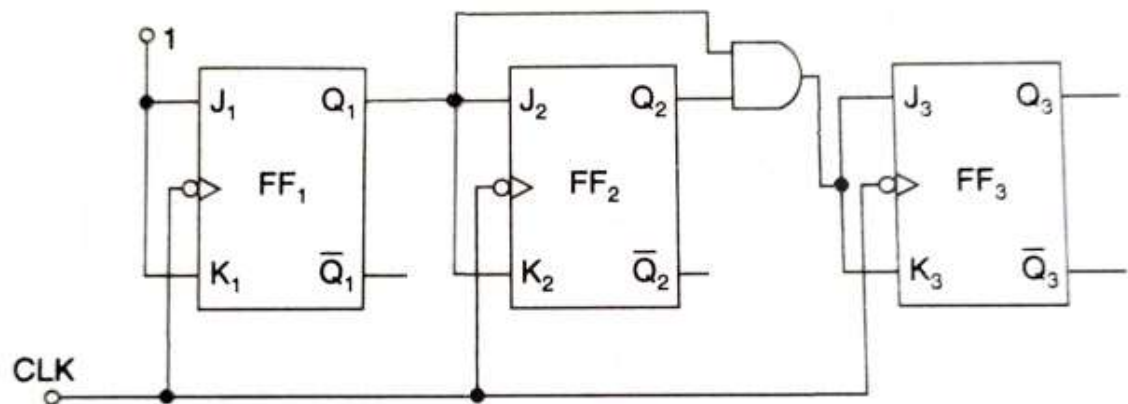
(b) Excitation table

Counter Design

Design of a Synchronous 3-bit Up Counter using J-K FFs:



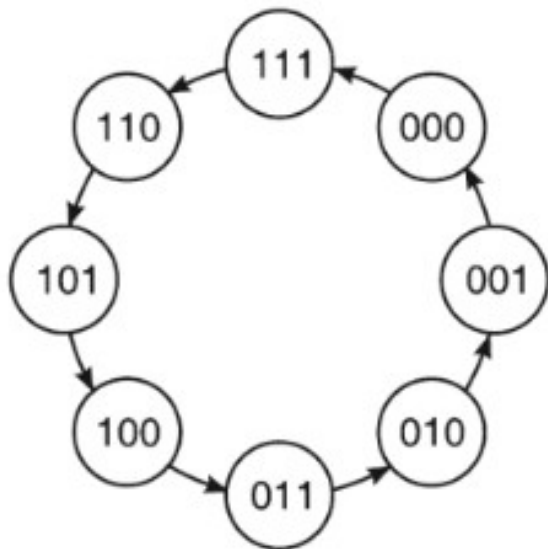
(c) Karnaugh maps for a 3-bit up counter using J-K FFs.



Logic diagram of the synchronous 3-bit up counter using J-K FFs.

Counter Design

Design of a Synchronous 3-bit Down Counter using J-K FFs:



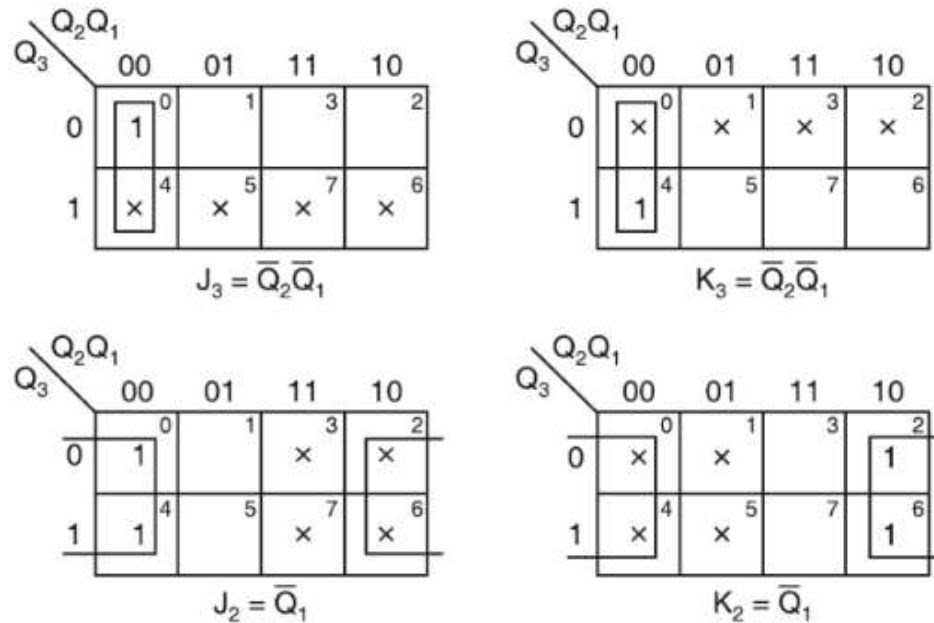
(a) State diagram

PS			NS			Required excitations					
Q_3	Q_2	Q_1	Q_3	Q_2	Q_1	J_3	K_3	J_2	K_2	J_1	K_1
0	0	0	1	1	1	1	x	1	x	1	x
1	1	1	1	1	0	x	0	x	0	x	1
1	1	0	1	0	1	x	0	x	1	1	x
1	0	1	1	0	0	x	0	0	x	x	1
1	0	0	0	1	1	x	1	1	x	1	x
0	1	1	0	1	0	0	x	x	0	x	1
0	1	0	0	0	1	0	x	x	1	1	x
0	0	1	0	0	0	0	x	0	x	x	1

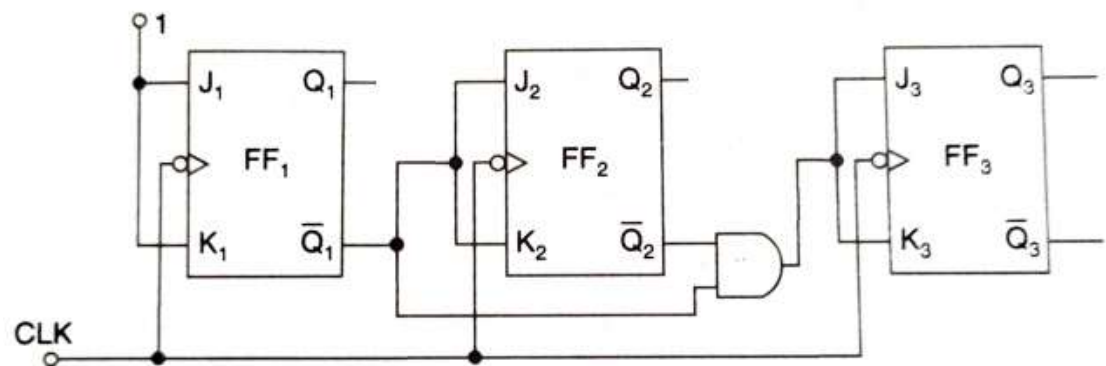
(b) Excitation table

Counter Design

Design of a Synchronous 3-bit Down Counter using J-K FFs:



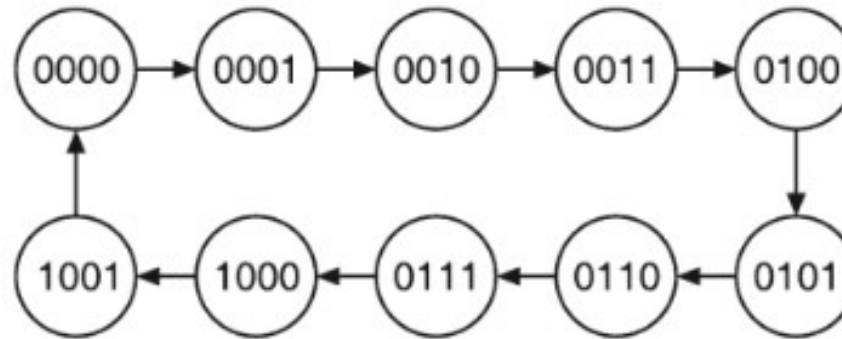
(c) Karnaugh maps for a 3-bit down counter using J-K FFs.



Logic diagram of the synchronous 3-bit down counter using J-K FFs.

Counter Design

Design of a Synchronous BCD Counter using J-K FFs:



(a) State diagram

PS				NS				Required excitations							
Q_4	Q_3	Q_2	Q_1	Q_4	Q_3	Q_2	Q_1	J_4	K_4	J_3	K_3	J_2	K_2	J_1	K_1
0	0	0	0	0	0	0	1	0	x	0	x	0	x	1	x
0	0	0	1	0	0	1	0	0	x	0	x	1	x	x	1
0	0	1	0	0	0	1	1	0	x	0	x	x	0	1	x
0	0	1	1	0	1	0	0	0	x	1	x	x	1	x	1
0	1	0	0	0	1	0	1	0	x	x	0	0	x	1	x
0	1	0	1	0	1	1	0	0	x	x	0	1	x	x	1
0	1	1	0	0	1	1	1	0	x	x	0	x	0	1	x
0	1	1	1	1	0	0	0	1	x	x	1	x	1	x	1
1	0	0	0	1	0	0	1	x	0	0	x	0	x	1	x
1	0	0	1	0	0	0	0	x	1	0	x	0	x	x	1

(b) Excitation table

Counter Design

Design of a Synchronous BCD Counter using J-K FFs:

$Q_4Q_3 \backslash Q_2Q_1$		00	01	11	10
		0	1	3	2
00					
01				1	
11		x	x	x	x
10		x	x	x	x

$$J_4 = Q_3Q_2Q_1$$

$Q_4Q_3 \backslash Q_2Q_1$		00	01	11	10
		0	1	3	2
00		x	x	x	x
01		x	x	x	x
11		x	x	x	x
10			1	x	x

$$K_4 = Q_1$$

$Q_4Q_3 \backslash Q_2Q_1$		00	01	11	10
		0	1	3	2
00				1	
01		x	x	x	x
11		x	x	x	x
10				x	x

$$J_3 = Q_2Q_1$$

$Q_4Q_3 \backslash Q_2Q_1$		00	01	11	10
		0	1	3	2
00			1	x	x
01			1	x	x
11		x	x	x	x
10				x	x

$$J_2 = \bar{Q}_4Q_1$$

$Q_4Q_3 \backslash Q_2Q_1$		00	01	11	10
		0	1	3	2
00		x	x	1	
01		x	x	1	
11		x	x	x	x
10		x	x	x	x

$$K_2 = Q_1$$

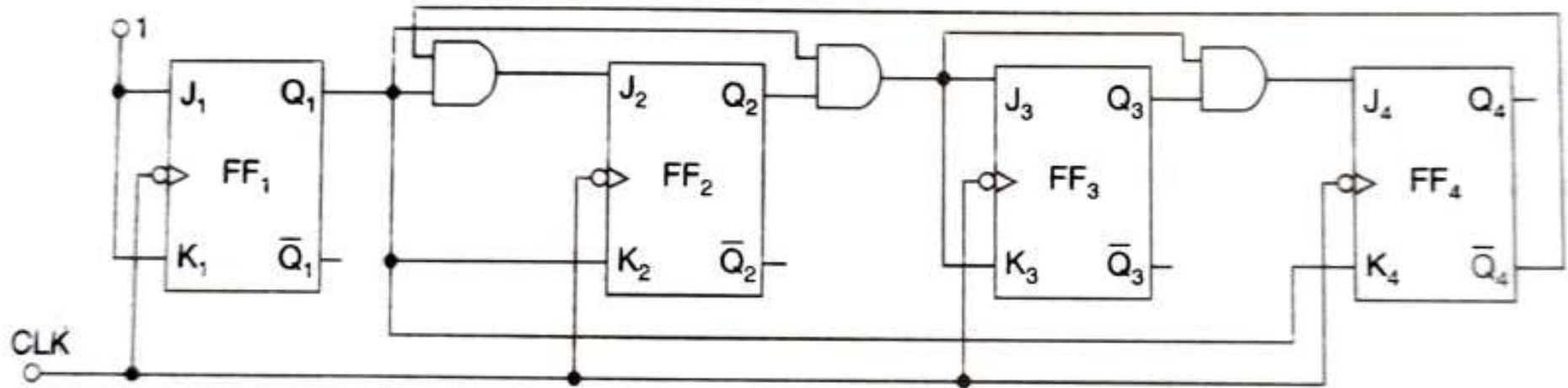
$Q_4Q_3 \backslash Q_2Q_1$		00	01	11	10
		0	1	3	2
00		x	x	x	x
01				1	
11		x	x	x	x
10		x	x	x	x

$$K_3 = Q_2Q_1$$

K-maps for excitations of synchronous BCD counter using J-K flip-flops

Counter Design

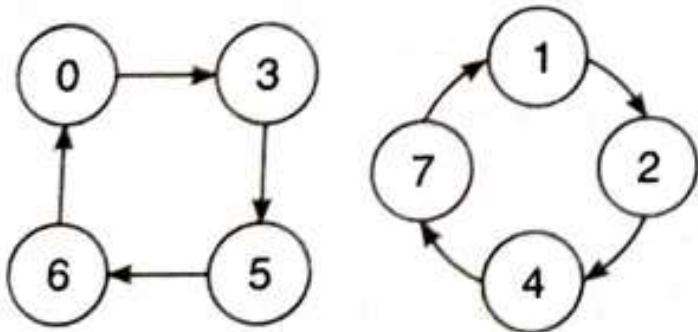
Design of a Synchronous BCD Counter using J-K FFs:



Logic diagram of synchronous BCD counter using J-K flip-flops.

Counter Design

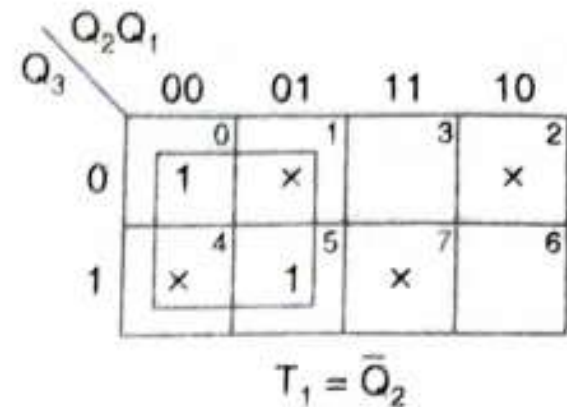
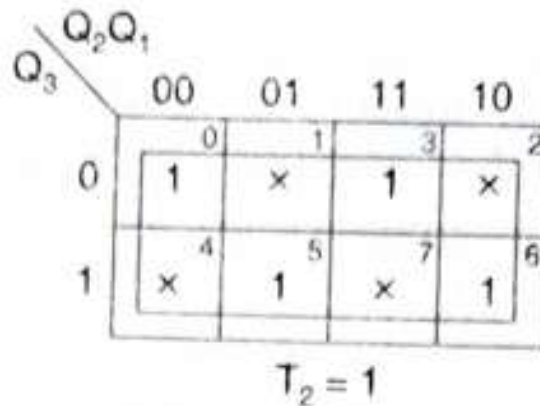
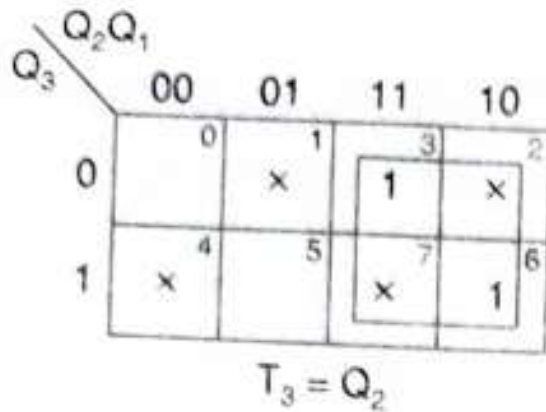
Design a type T counter that goes through states 0, 3, 5, 6, 0... :



(a) State diagram

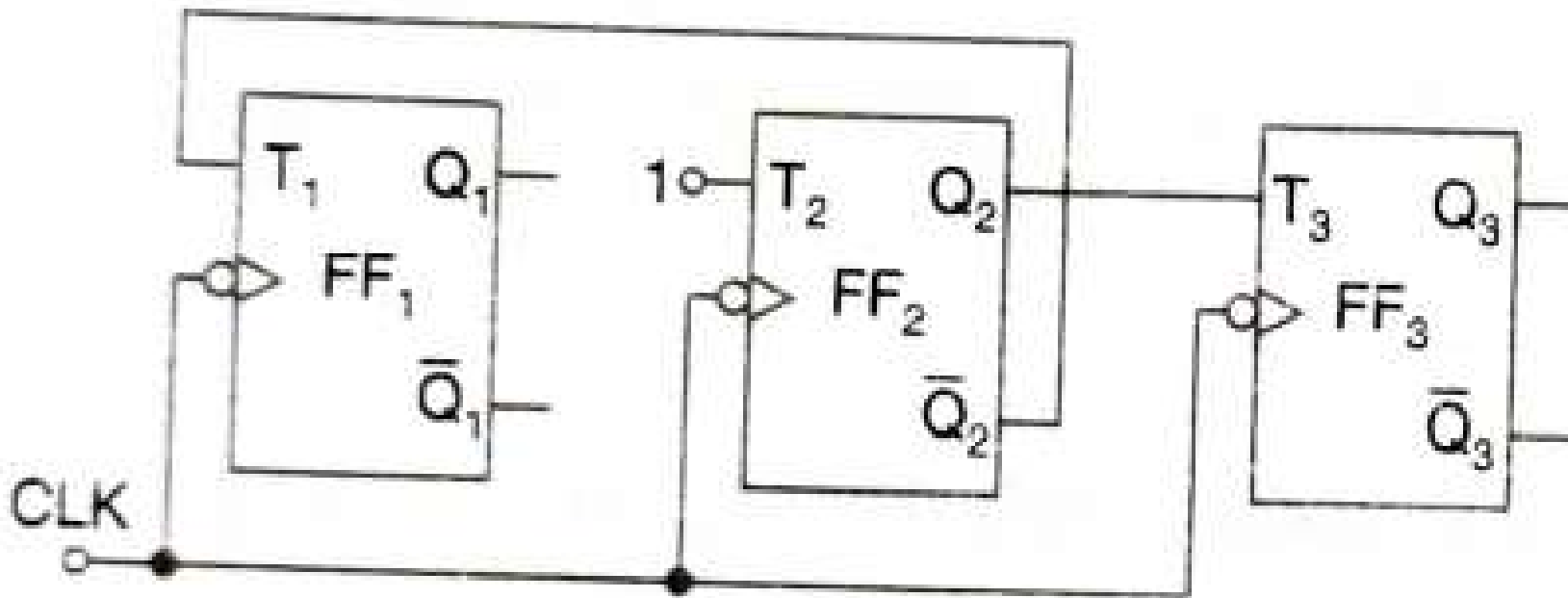
PS			NS			Required excitations		
Q_3	Q_2	Q_1	Q_3	Q_2	Q_1	T_3	T_2	T_1
0	0	0	0	1	1	0	1	1
0	1	1	1	0	1	1	1	0
1	0	1	1	1	0	0	1	1
1	1	0	0	0	0	1	1	0

(b) Excitation table



Counter Design

Design a type T counter that goes through states 0, 3, 5, 6, 0... :



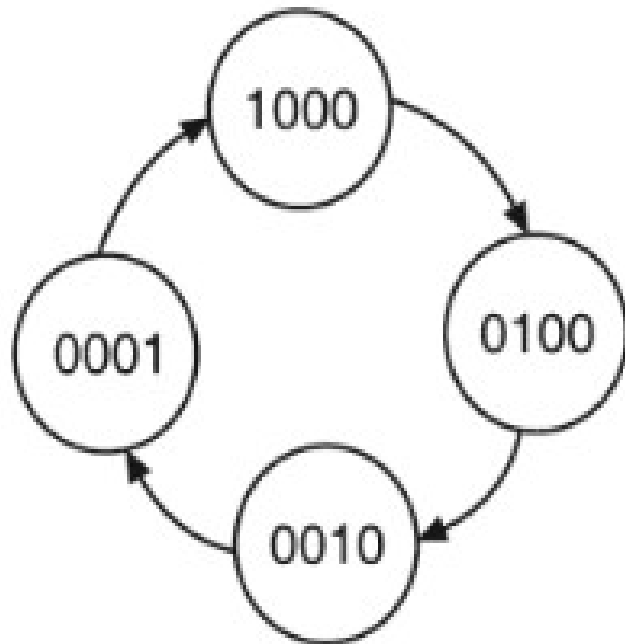
Counter Design

Ring Counter:

- A *ring counter* is a circular shift register with only one flip-flop being set at any particular time; all others are cleared.
- The single bit is shifted from one flip-flop to the next to produce the sequence of timing signals.
- The FFs are arranged as in a normal shift register, i.e. the Q output of each stage is connected to the D input of the next stage, but the Q output of the last FF is connected back to the D input of the first FF.
- The number of distinct states in the ring counter , i.e. the mod of the ring counter is equal to the number of FFs used in the counter.
- An n -bit ring counter can count only n bits, whereas an n -bit ripple counter can count 2^n bits.

Counter Design

Ring Counter:



(a) State diagram

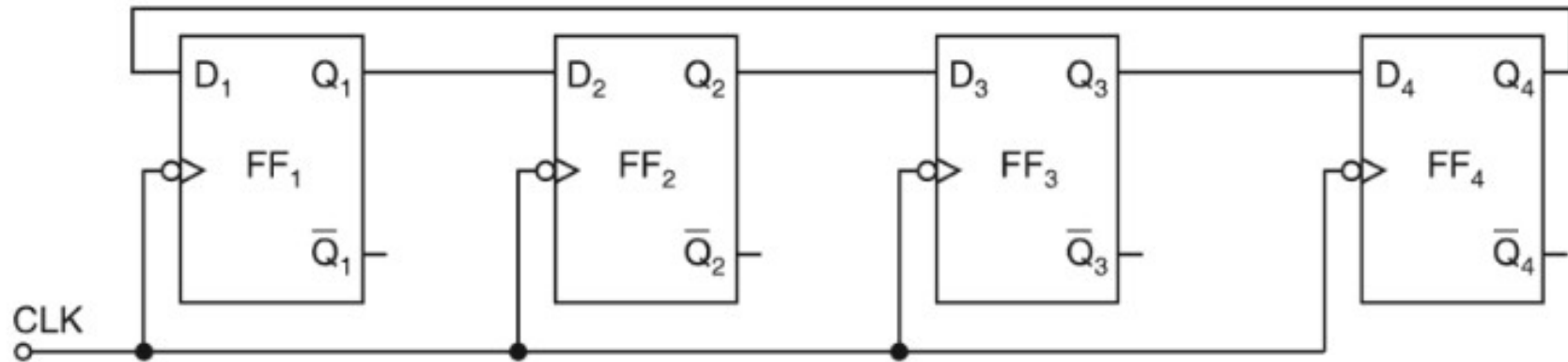
Q_1	Q_2	Q_3	Q_4	After clock pulse
1	0	0	0	0
0	1	0	0	1
0	0	1	0	2
0	0	0	1	3
1	0	0	0	4
0	1	0	0	5
0	0	1	0	6
0	0	0	1	7

(b) Sequence table

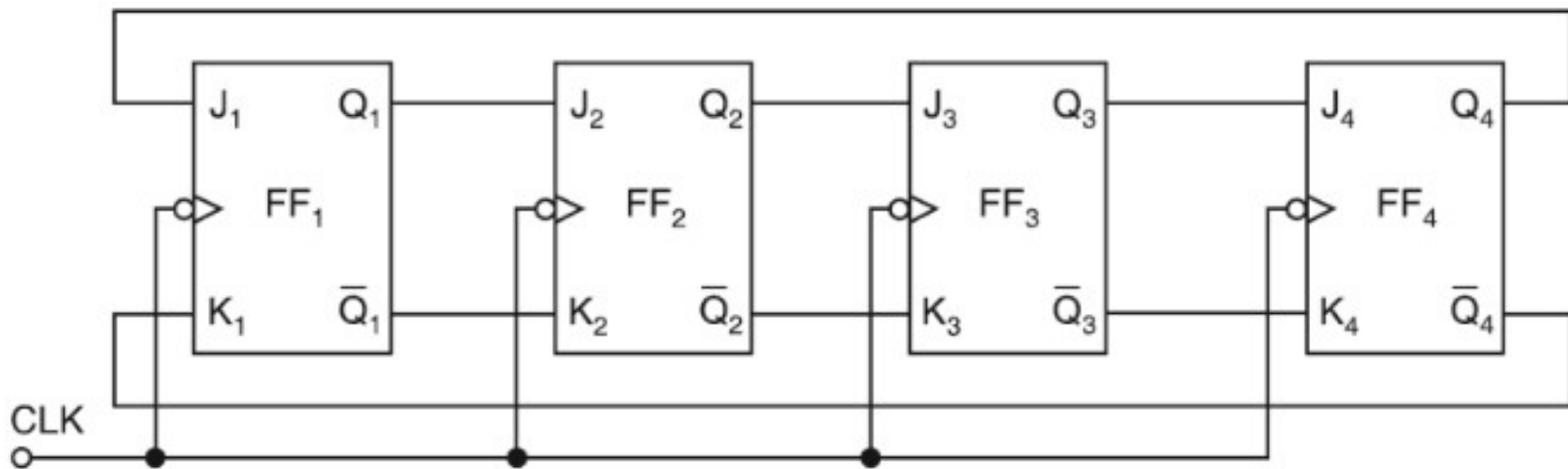
State diagram and sequence table of a 4-bit ring counter.

Counter Design

Ring Counter:



Logic diagram of a 4-bit ring counter using D flip-flops.



Logic diagram of a 4-bit ring counter using J-K flip-flops.

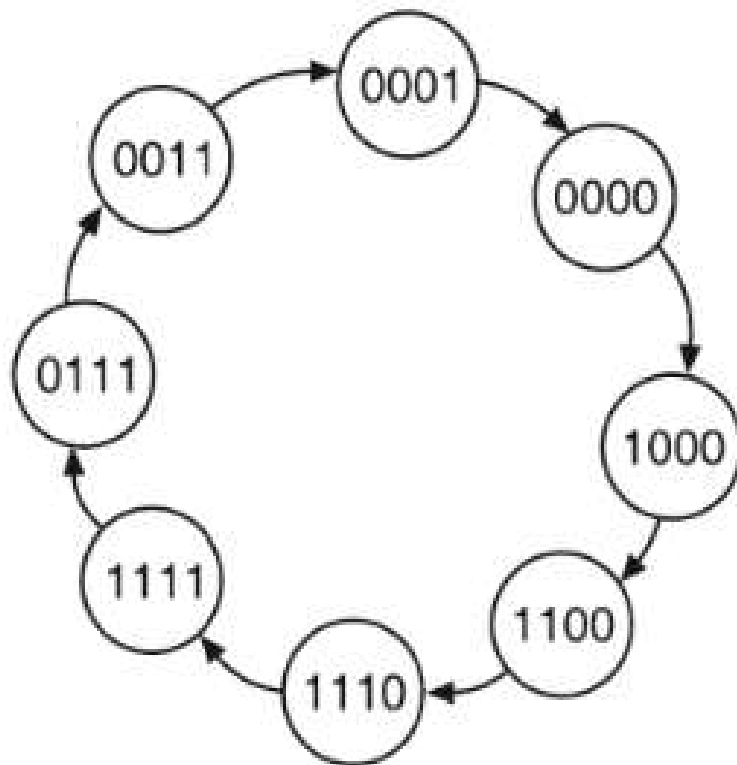
Counter Design

Johnson Counter:

- ❖ A k -bit ring counter circulates a single bit among the flip-flops to provide k distinguishable states.
- ❖ The number of states can be doubled if the shift register is connected as a switch-tail ring counter / Johnson counter.
- ❖ A switch-tail ring counter is a circular shift register with the complemented output of the last flip-flop connected to the input of the first flip-flop.
- ❖ This counter is obtained from a serial-in, serial-out shift register by providing feedback from the inverted output of the last FF to the D input of the first FF.
- ❖ The Q output of each stage is connected to the D input of the next stage, but the $\overline{Q}$ output of the last stage is connected to the D input of first stage.
- ❖ An n FF Johnson counter can have $2n$ unique states and can count up to $2n$ pulses. So, it is a mod- $2n$ counter.

Counter Design

Johnson Counter:



(a) State diagram

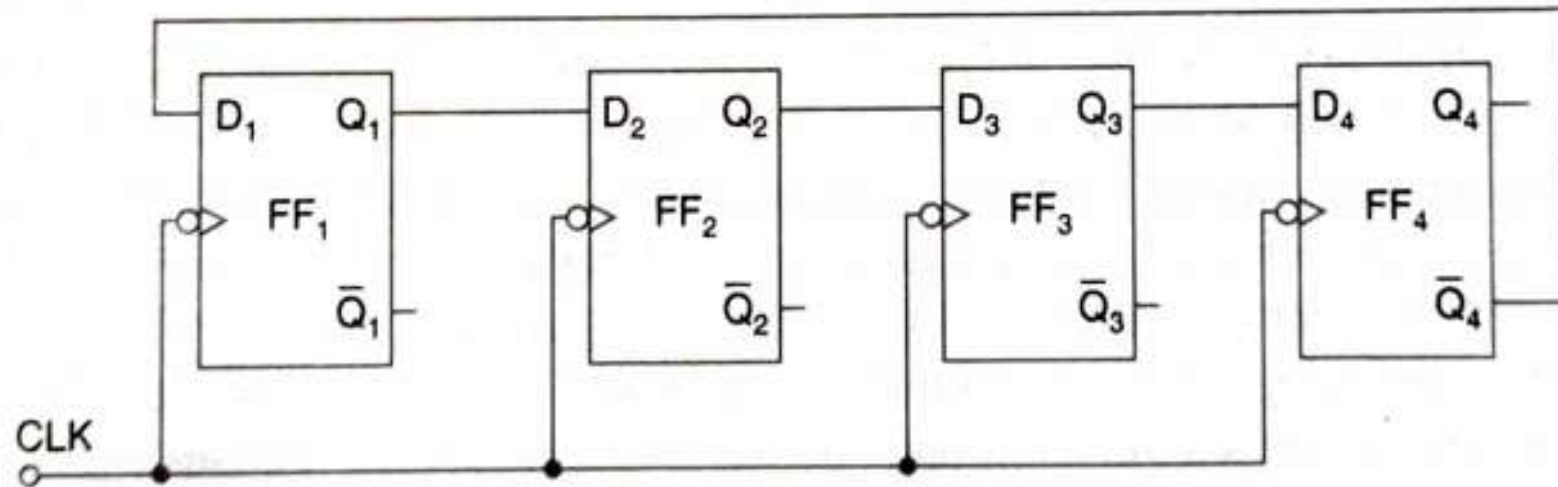
Q_1	Q_2	Q_3	Q_4	After clock pulse
0	0	0	0	0
1	0	0	0	1
1	1	0	0	2
1	1	1	0	3
1	1	1	1	4
0	1	1	1	5
0	0	1	1	6
0	0	0	1	7
0	0	0	0	8
1	0	0	0	9

(b) Sequence table

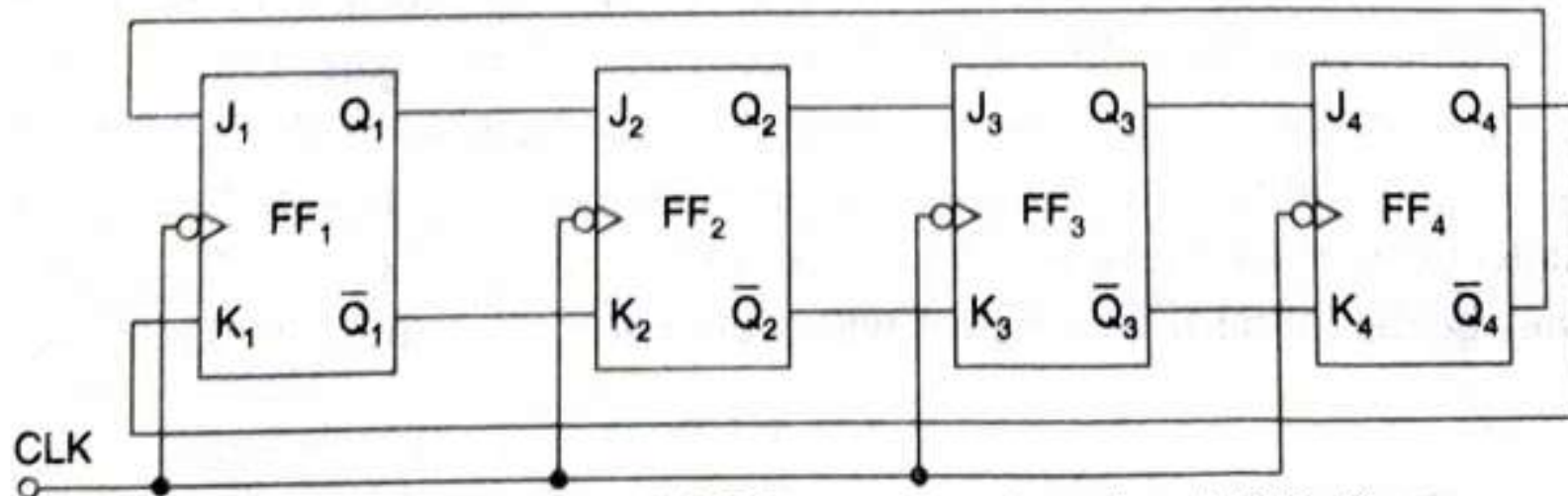
State diagram and sequence table of a Johnson counter

Counter Design

Johnson Counter:



Logic diagram of a 4-bit Johnson counter using D flip-flops.



Logic diagram of a 4-bit Johnson counter using J-K flip-flops.

Frequency Division

Binary Counter as a Frequency Divider:

- ✓ A **binary counter** can be used as a **frequency divider** by reducing the frequency of an input clock signal. This is widely used in digital circuits such as **clock generation, frequency synthesis, and digital timers**.

How a Binary Counter Works as a Frequency Divider:

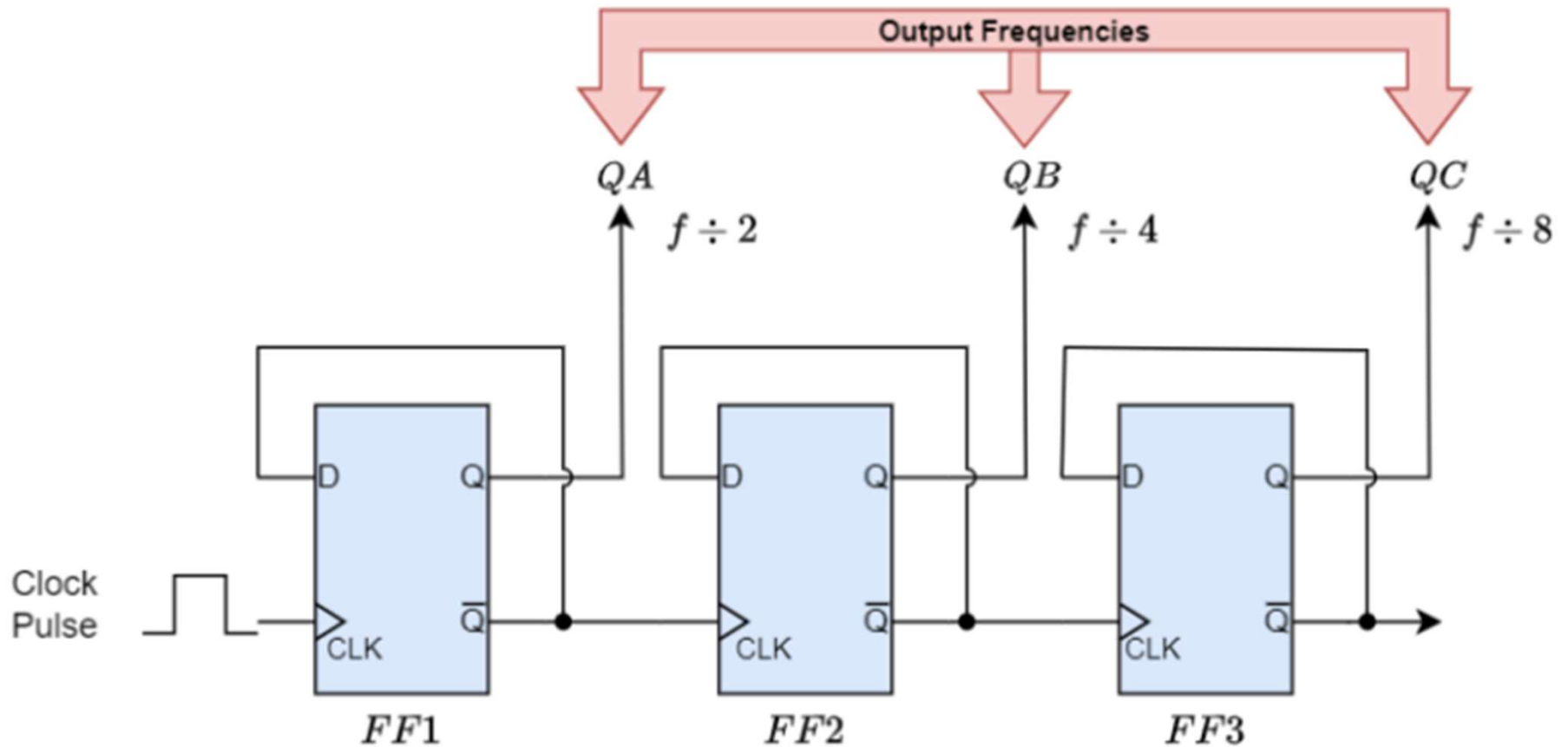
- ✓ A **binary counter** consists of **flip-flops**, where each **flip-flop toggles its state at half the frequency of the previous one**. This means that each stage of the counter **divides the input clock frequency by 2**.
- ✓ Example: 4-bit Binary Counter as a Frequency Divider

Flip-Flop (Stage)	Frequency Division Factor	Output Frequency (f_{out})
FF1	$f_{in} \div 2$	$f_{out1} = f_{in}/2$
FF2	$f_{in} \div 4$	$f_{out2} = f_{in}/4$
FF3	$f_{in} \div 8$	$f_{out3} = f_{in}/8$
FF4	$f_{in} \div 16$	$f_{out4} = f_{in}/16$

- If the input clock frequency is 1 MHz, then The **first flip-flop** produces 500 kHz, The **second flip-flop** produces 250 kHz, The **third flip-flop** produces 125 kHz, The **fourth flip-flop** produces 62.5 kHz.

Frequency Division

Binary Counter as a Frequency Divider:



Frequency Division

Advantages of Using Binary Counters as Frequency Dividers:

- ✓ Simple and efficient frequency division
- ✓ No need for external components
- ✓ Low power consumption in CMOS implementation
- ✓ Used in clock generation for microcontrollers and digital circuits

Applications of Frequency Dividers:

- ✓ Clock generation in microprocessors
- ✓ Timing circuits in digital systems
- ✓ Frequency synthesizers in communication systems
- ✓ Pulse generation for ADC/DAC circuits

Frequency Division

Frequency Division Hazards:

- ✓ **Frequency division hazards** occur when a **binary counter** or **frequency divider circuit** produces incorrect or unstable output due to **timing mismatches, glitches, or race conditions**. These hazards can cause **incorrect frequency division**, unwanted pulses, or incorrect counting sequences.

Types of Hazards in Frequency Division:

1. Glitches Due to Propagation Delay

✓ In **asynchronous (ripple) counters**, each flip-flop depends on the previous one. Due to **propagation delay**, all flip-flops do not change state **simultaneously**, causing temporary incorrect outputs (glitches).

✓ **Solution:** Use **synchronous counters**, where all flip-flops are triggered by the **same clock pulse**.

2. Meta-Stability Hazard

✓ Occurs when a **flip-flop's setup and hold time requirements** are not met, causing an unpredictable output.

✓ **Solution:** Ensure proper **setup and hold time** requirements are met by using **synchronizers** or adding **buffer stages**.

Frequency Division

3. Race Condition Hazard

- ✓ When multiple signals change at the same time, but due to **different propagation delays**, the circuit produces incorrect results.
- ✓ **Solution:** Minimize race conditions by **using synchronous designs** and **adding delay buffers**.

4. Clock Skew Hazard

- ✓ In a **large counter circuit**, the clock signal may not reach all flip-flops at exactly the same time, leading to incorrect output transitions.
- ✓ **Solution:** Use **clock distribution networks** like **H-tree architecture** to minimize skew.

5. Power Supply Noise Hazard

- ✓ Fast switching of flip-flops can cause **voltage fluctuations**, affecting the correct toggling of flip-flops.
- ✓ **Solution:** Use **decoupling capacitors** to stabilize the power supply.

How to Avoid Frequency Division Hazards?

- ✓ Use Synchronous Counters instead of ripple counters.
- ✓ Ensure Setup and Hold Times for flip-flops.
- ✓ Minimize Propagation Delays by using high-speed logic families.
- ✓ Use Proper Clock Distribution to prevent clock skew.
- ✓ Add Decoupling Capacitors to reduce power supply noise.

Session 12

Lecture: Registers

- ✓ Registers
- ✓ Shift Registers
- ✓ LIFO
- ✓ FIFO
- ✓ LFSR

Registers

- ❖ A **register** is a group of **flip-flops** used to store and transfer binary data. It is a **sequential circuit** that holds data temporarily and is widely used in microprocessors, memory, and data storage applications.
- ◆ Each flip-flop stores **1-bit**, so an **n-bit register** consists of **n flip-flops**.
- ◆ Controlled by a **clock signal** to enable data storage and movement.

Types of Registers:-

1. Buffer Register (Simple Register)

- ✓ Stores a single binary word (set of bits) temporarily
- ✓ No data movement, only storage
- ✓ Used in temporary data storage inside microprocessors

2. Shift Register

- ✓ Moves data from one flip-flop to another on each clock pulse
- ✓ Used for serial data transfer and delay operations

Registers

Types of Shift Registers:

Serial-In Serial-Out (SISO) – Data shifts one bit at a time in and out.

Serial-In Parallel-Out (SIPO) – Data shifts in serially and is output in parallel.

Parallel-In Serial-Out (PISO) – Data is loaded in parallel and shifted out serially.

Parallel-In Parallel-Out (PIPO) – Data is loaded and output in parallel.

Applications: Serial-to-parallel conversion, data buffering, communication systems.

3. Universal Shift Register

- ✓ Can perform all shift register operations (SISO, SIPO, PISO, PIPO)
- ✓ Bi-directional data movement (left/right shifting)
- ✓ Used in multiprocessors, arithmetic operations, and encryption systems

4. Counter Register

- ✓ Increments or decrements data on each clock pulse
- ✓ Used in program counters, memory addressing, and event counting

5. Ring Counter

- ✓ A shift register where the last flip-flop's output is fed back to the first
- ✓ Only one flip-flop is active at a time
- ✓ Used in sequence generation and timing circuits

Registers

6. Johnson Counter (Twisted Ring Counter):

- ✓ A ring counter with inverted feedback
- ✓ Counts twice the number of states as a standard ring counter
- ✓ Used in pattern generation, frequency division, and state machines

Comparison of Different Registers:

Register Type	Data Movement	Input Type	Output Type	Applications
Buffer Register	No movement	Parallel	Parallel	Temporary storage
Shift Register	Left/Right shift	Serial/Parallel	Serial/Parallel	Serial data communication
Universal Register	Left/Right shift	Serial/Parallel	Serial/Parallel	Multipurpose storage
Counter Register	Increments/Decrements	Parallel	Parallel	Program counters, memory addressing
Ring Counter	Cyclic shift	Serial	Serial	Sequence generation
Johnson Counter	Inverted shift	Serial	Serial	Timing applications

Registers

Applications of Registers:

- ✓ Microprocessors and microcontrollers (storing and processing data)
- ✓ Data transfer and communication (serial-to-parallel conversion)
- ✓ Memory addressing and storage
- ✓ Counters and timers
- ✓ Cryptographic operations

Shift Registers:

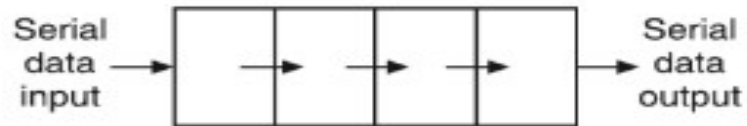
- ❖ A **shift register** is a **sequential circuit** that **stores and shifts binary data** in a series of **flip-flops**. It is commonly used for **data storage, data transfer, and delay operations** in digital systems.
- ◆ **Each flip-flop stores 1 bit of data**
- ◆ **Data shifts on every clock pulse**
- ◆ **Used in serial-to-parallel and parallel-to-serial conversion**

Registers

Types of Shift Registers:

- Data may be shifted into or out of the register either in serial form or in parallel form.
- So, there are four basic types of shift registers:
 - Serial-in, Serial-out (SISO)
 - Serial-in, Parallel-out (SIPO)
 - Parallel-in, Serial-out (PISO)
 - Parallel-in, Parallel-out (PIPO)
- Shift register has two shift operations that are controlled by the signals *SHR* (Shift Right) and *SHL* (Shift Left).
- In addition to the shift operations, a register may also have the capability of the rotate operations *ROR* (Rotate Right) and *ROL* (Rotate Left).

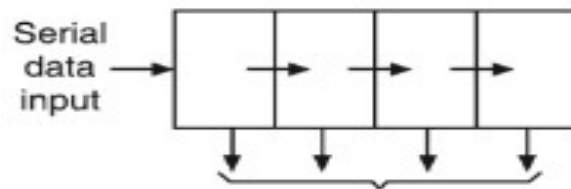
Registers



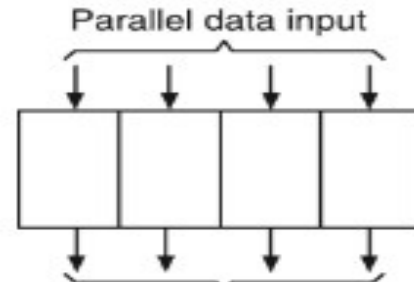
(a) Serial-in, serial-out, shift-right, shift register



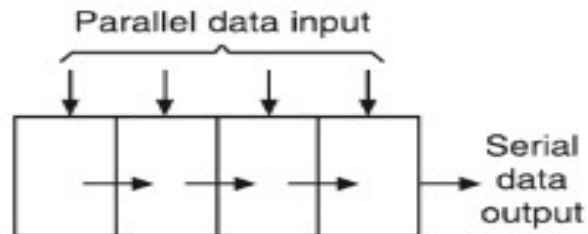
(b) Serial-in, serial-out, shift-left, shift register



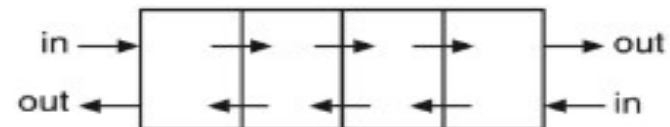
(c) Serial-in, parallel out, shift register



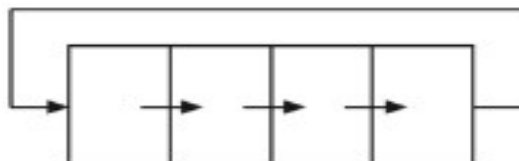
(d) Parallel-in, parallel-out, shift register



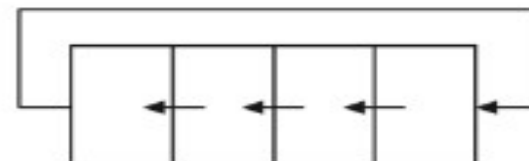
(e) Parallel-in, serial-out, shift register



(f) Serial-in, serial-out, shift-left, shift-right, (bidirectional) shift register



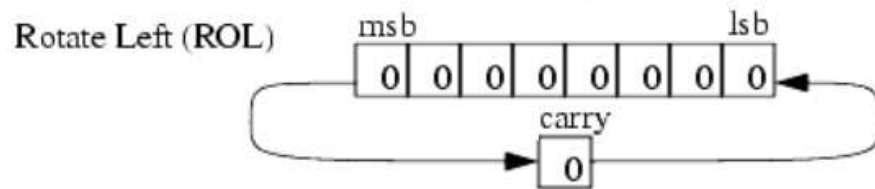
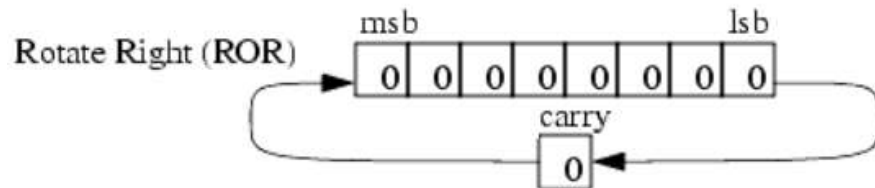
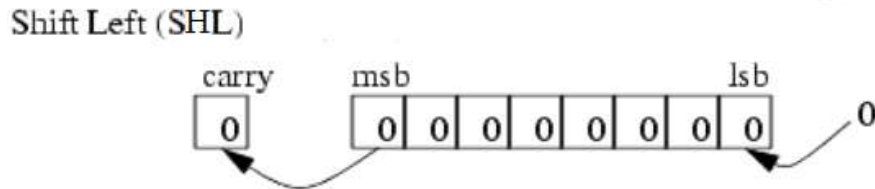
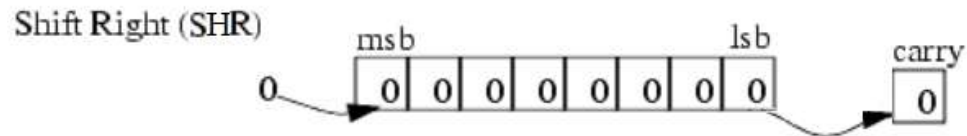
(g) Rotate-right shift register



(h) Rotate-left shift register

Data transfer in registers.

Registers



MSB							LSB
1	0	1	0	1	1	0	0

<i>SHR</i> 1	0	1	0	1	0	1	1	0
--------------	---	---	---	---	---	---	---	---

<i>SHL</i> 1	0	1	0	1	1	0	0	0
--------------	---	---	---	---	---	---	---	---

<i>ROR</i> 1	0	1	0	1	0	1	1	0
--------------	---	---	---	---	---	---	---	---

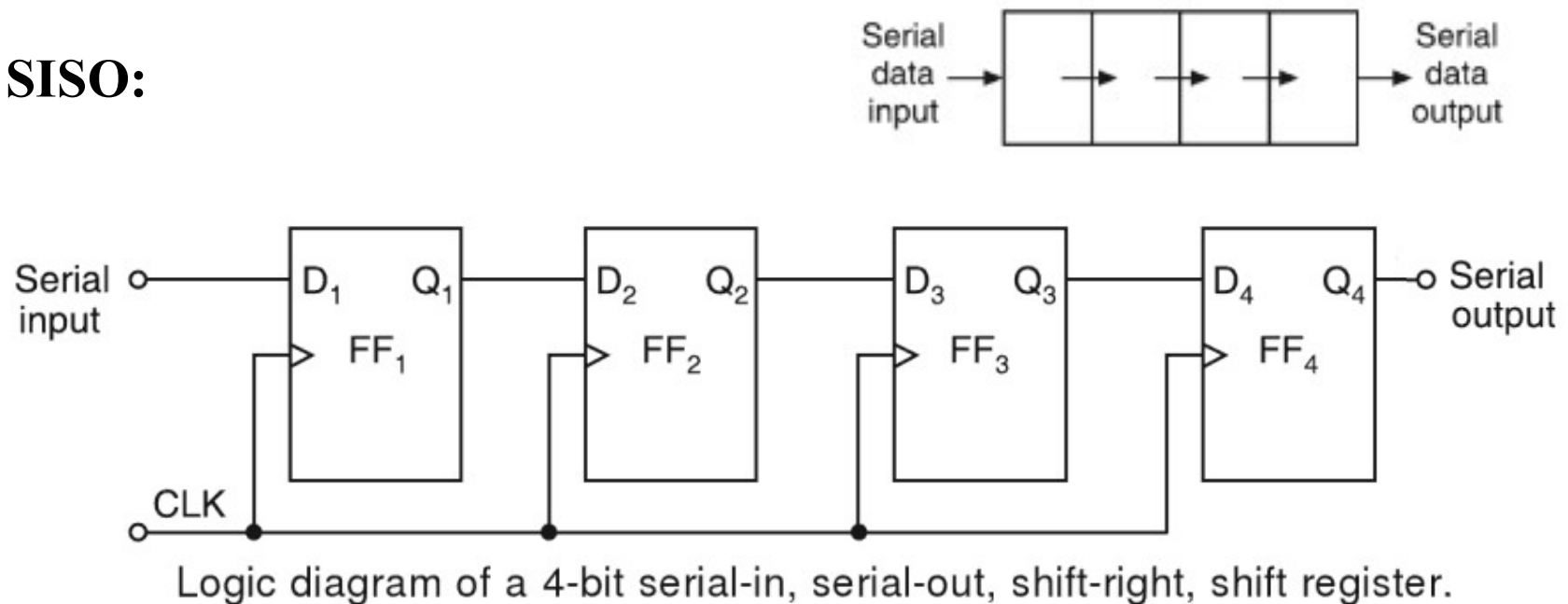
<i>ROL</i> 1	0	1	0	1	1	0	0	1
--------------	---	---	---	---	---	---	---	---

Registers

Serial-in, Serial-out (SISO):

- ✓ A **Serial-In, Serial-Out (SISO) shift register** is a **sequential circuit** that shifts data **one bit at a time** from input to output in a **serial manner**. It is mainly used for **temporary data storage, data delay, and serial communication**.
- ✓
 - ◆ Data is entered one bit at a time (serial input)
 - ◆ Data is shifted through flip-flops on each clock pulse
 - ◆ Data is output one bit at a time (serial output)

4-bit SISO:



Registers

4-bit SISO:

Clock Pulse	Data	D ₁	D ₂	D ₃	D ₄	Q ₁	Q ₂	Q ₃	Q ₄
	1010								
1st	LSB	0				0			
2nd	Data	1	0			1	0		
3rd	In	0	1	0		0	1	0	
4th	MSB	1	0	1	0	1	0	1	0
5th			1	0	1		1	0	1
6th				1	0			1	0
7th					1				1

LSB
 Data Out
 MSB

- In n -bit serial-in serial-out (SISO) shift register n number of clock pulses are required to store the n -bit data but in $(2n-1)$ th clock pulse the n -bit data out from the shift register.

Applications of SISO Shift Registers:

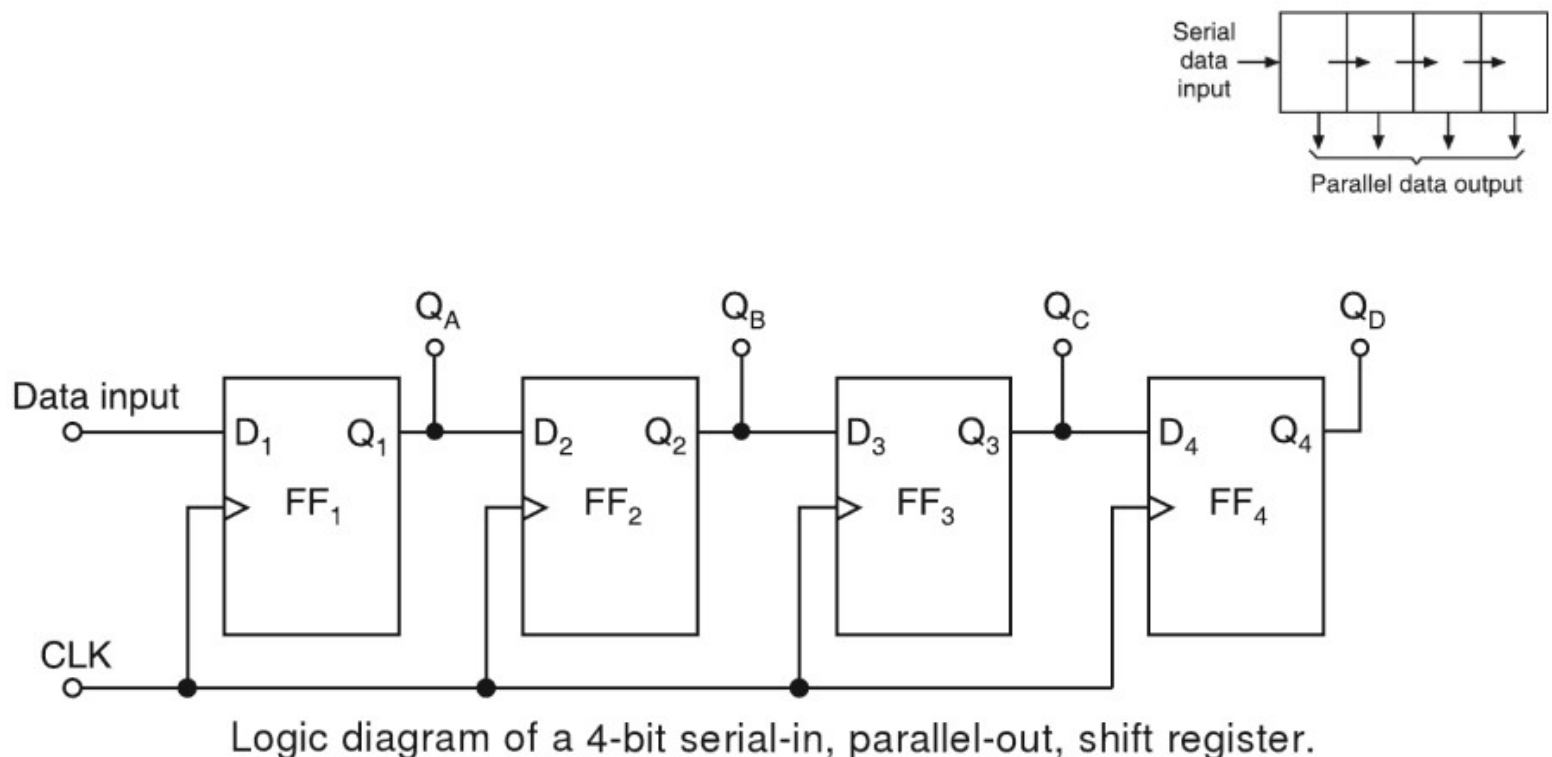
- ✓ Data buffering in serial communication (UART, SPI, I2C)
- ✓ Digital signal processing (DSP)
- ✓ Time delay circuits
- ✓ Data serialization and deserialization

Registers

Serial-in, Parallel-out (SIPO):

- A **Serial-In, Parallel-Out (SIPO) shift register** is a **sequential circuit** that takes in **serial data (one bit at a time)** and then outputs it **in parallel after multiple clock cycles**. It is used for **serial-to-parallel data conversion** in communication systems.
- - ◆ Data is input serially (one bit per clock cycle)
 - ◆ Each flip-flop stores a bit and shifts it on every clock pulse
 - ◆ After "n" clock pulses, all outputs are available in parallel

4-bit SIPO:



Registers

4-bit SIPO:

Clock Pulse	Data	D ₁	D ₂	D ₃	D ₄	Q ₁	Q ₂	Q ₃	Q ₄
	1010								
1st	LSB	0				0			
2nd	Data	1	0			1	0		
3rd	In	0	1	0		0	1	0	
4th	MSB	1	0	1	0	1	0	1	0

MSB

LSB

Data Out

- In n -bit serial-in parallel-out (SIPO) shift register n number of clock pulses are required to store the n -bit data but in n th clock pulse the n -bit data out from the shift register.

Applications of SIPO Shift Registers:

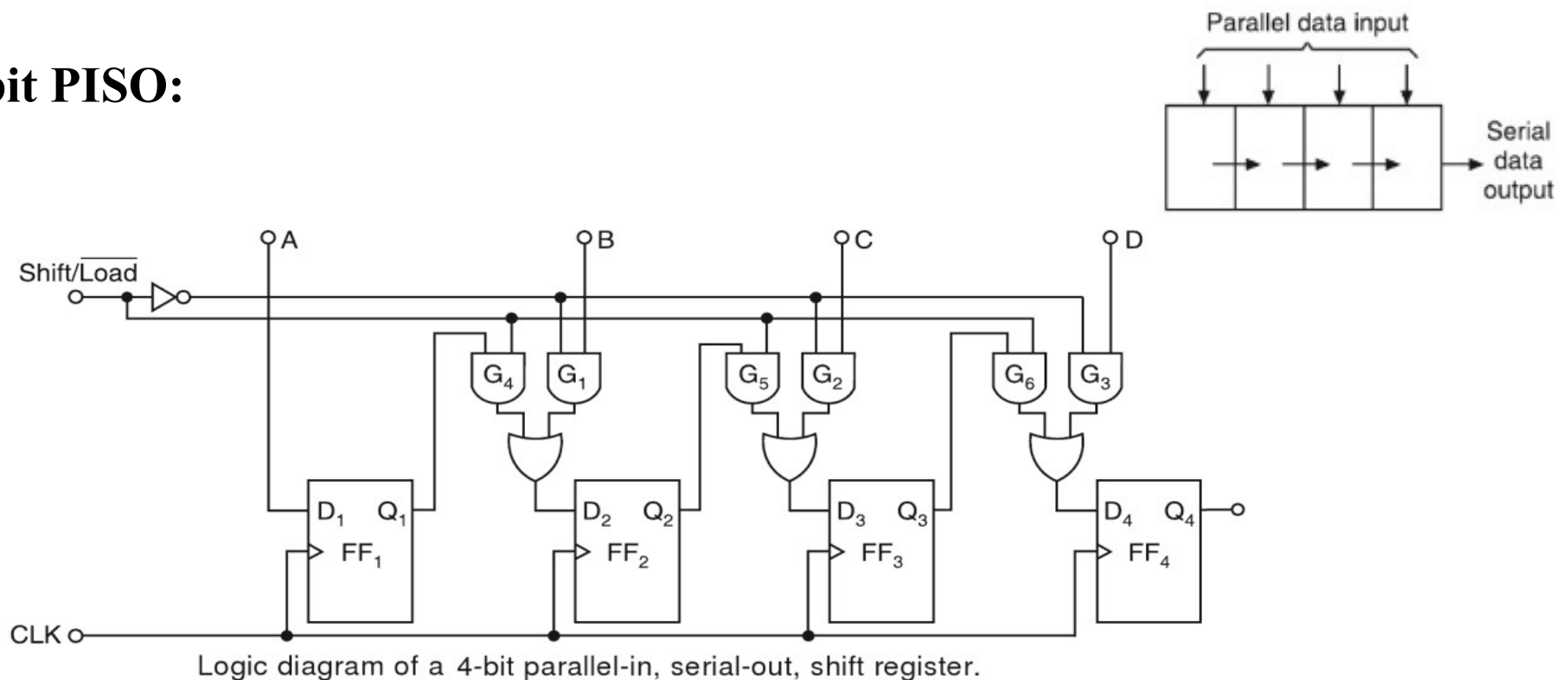
- ✓ Serial-to-parallel data conversion (UART, SPI, I2C)
- ✓ LED display controllers (e.g., 74HC595 shift register for LED matrix)
- ✓ Data transmission and reception in microcontrollers
- ✓ Expanding microcontroller I/O ports

Registers

Parallel-in, Serial-out (PISO):

- A **Parallel-In, Serial-Out (PISO) shift register** is a **sequential circuit** that takes in **parallel data (multiple bits at once)** and then shifts it out **serially (one bit at a time)**. It is used for **parallel-to-serial data conversion** in communication systems.
- - ◆ Multiple bits are loaded in parallel at once
 - ◆ The data is shifted out bit-by-bit on each clock pulse
 - ◆ Used in serial communication and data transmission

4-bit PISO:



Registers

4-bit PISO:

- ✓ The signal $\text{Shift}/\overline{\text{Load}}$ allows (a) the data to be entered in parallel form into the register and (b) the data to be shifted out serially from terminal Q_4 .
- ✓ When $\text{Shift}/\overline{\text{Load}}$ line is HIGH, gates G_1 , G_2 , and G_3 are disabled, but gates G_4 , G_5 , and G_6 are enabled allowing the bits to shift-right from one stage to the next.
- ✓ When $\text{Shift}/\overline{\text{Load}}$ line is LOW, gates G_4 , G_5 , and G_6 are disabled, but whereas gates G_1 , G_2 , and G_3 are enabled allowing the data input to appear at the D inputs of the respective FFs.
- ✓ When a clock pulse is applied, these data bits are shifted to the Q output terminals of the FFs and, therefore, data is inputted in one step.
- ✓ The OR gate allows either the normal shifting operation or the parallel data entry depending on which AND gates are enabled by the level on the $\text{Shift}/\overline{\text{Load}}$ input.

Registers

4-bit PISO:

Clock Pulse	Shift /Load	Data	D ₁	D ₂	D ₃	D ₄	Q ₁	Q ₂	Q ₃	Q ₄
		1010								
1st	0		1	0	1	0	1	0	1	0
2nd	1		MSB Data In LSB					1	0	1
3rd	1								1	0
4th	1									1

LSB

 Data Out

 MSB

- In n -bit parallel-in serial-out (PISO) shift register a **single** clock pulse is required to store the n -bit data but in **n th** clock pulse n -bit data out from the shift register.

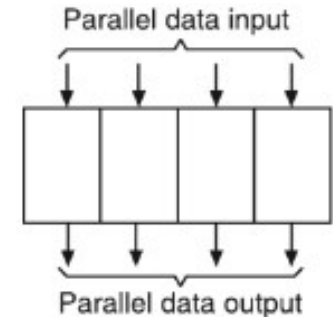
Applications of PISO Shift Registers:

- ✓ Parallel-to-serial data conversion (UART, SPI, I2C)
- ✓ Data transmission in microcontrollers
- ✓ Multiplexing of parallel data for communication systems
- ✓ Reducing I/O pin requirements in digital circuits

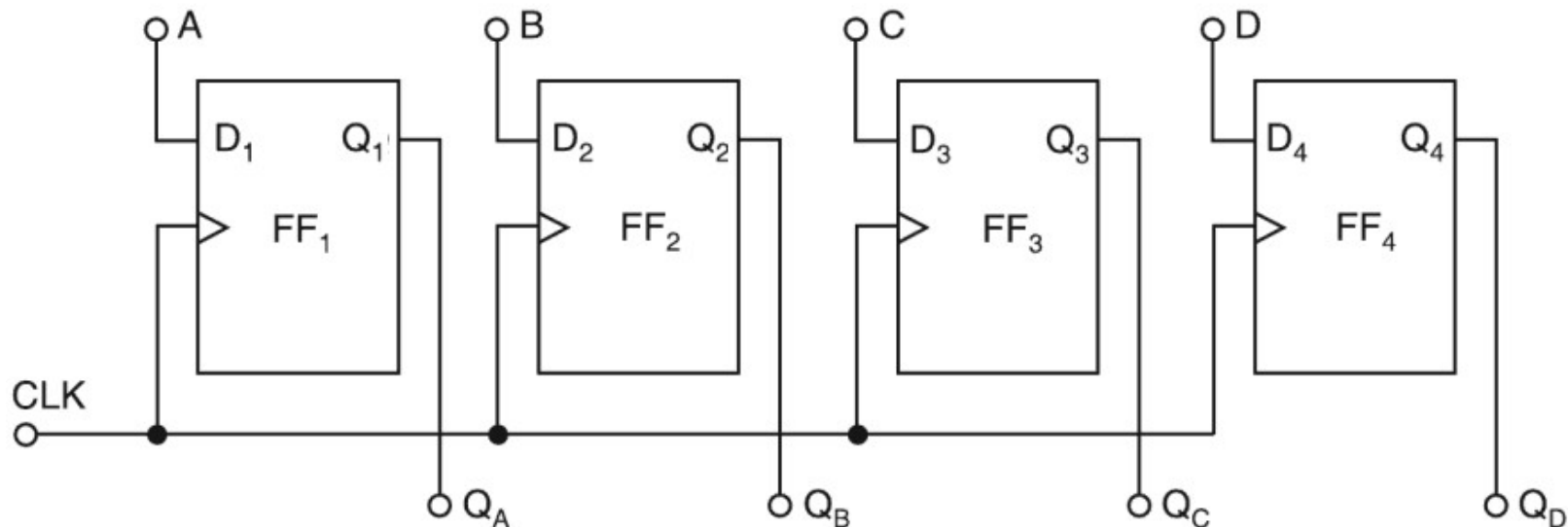
Registers

Parallel-in, Parallel-out (PIPO):

- A **Parallel-In, Parallel-Out (PIPO) shift register** is a **sequential circuit** that loads multiple bits **simultaneously in parallel** and outputs all bits **in parallel without shifting**.
- - ◆ Data is loaded into all flip-flops at once
 - ◆ The data remains until the next load operation
 - ◆ No shifting occurs, only storage and retrieval



4-bit PIPO:



Logic diagram of a 4-bit parallel-in, parallel-out, shift register.

Registers

4-bit PIPO:

Clock Pulse	Data	D ₁	D ₂	D ₃	D ₄	Q ₁	Q ₂	Q ₃	Q ₄
	1010								
1st		1	0	1	0	1	0	1	0

MSBLSBMSBLSB

Data InData Out

- In n -bit parallel-in parallel-out (PIPO) shift register a **single** clock pulse is required to store the n -bit data but in **same** clock pulse the n -bit data out from the shift register.

Applications of PIPO Shift Registers:

- ✓ Temporary data storage in processors and microcontrollers
- ✓ Parallel data transfer between digital systems
- ✓ Memory buffering for high-speed computing
- ✓ Multiprocessor communication

Registers

LIFO:

- ✓ A **LIFO (Last-In, First-Out) Register** is a type of **sequential memory structure** that stores and retrieves data in a **stack-like manner**. It follows the **LIFO principle**, meaning **the last data entered is the first data to be removed**.
- ◆ Data is pushed (stored) into the register sequentially
- ◆ Data is popped (retrieved) in reverse order
- ◆ Uses a stack pointer to track the top of the stack

Working Principle of a LIFO Register:

- A **LIFO register operates as a stack** with two primary operations:
 1. **Push Operation (Store Data)**
 1. Data is stored in memory **from bottom to top**.
 2. The stack pointer is incremented after each push.
 2. **Pop Operation (Retrieve Data)**
 1. The most recently stored data is retrieved first.
 2. The stack pointer is decremented after each pop.

Example:

Push Sequence: $A \rightarrow B \rightarrow C \rightarrow D$

Pop Sequence: $D \rightarrow C \rightarrow B \rightarrow A$

Registers

LIFO Register Implementation Using Flip-Flops:

- ✓ Typically implemented using D flip-flops or SR flip-flops
- ✓ A stack pointer register keeps track of the topmost data
- ✓ Requires control logic to handle push and pop operations

Truth Table Example for a 4-bit LIFO Register:

Clock Cycle	Operation	Stack Content (Bottom → Top)	Top of Stack Output
1st Pulse	Push A	A	A
2nd Pulse	Push B	A B	B
3rd Pulse	Push C	A B C	C
4th Pulse	Push D	A B C D	D
5th Pulse	Pop	A B C	C
6th Pulse	Pop	A B	B

Applications of LIFO Registers

- ✓ Stack memory in processors (e.g., call stack in microcontrollers)
- ✓ Expression evaluation (Postfix, Prefix notation processing)
- ✓ Undo operations in software (e.g., text editors, CAD tools)
- ✓ Recursive function execution in programming

Registers

FIFO:

- A **FIFO (First-In, First-Out) Register** is a type of **sequential memory structure** that stores and retrieves data in a **queue-like manner**. It follows the **FIFO principle**, meaning **the first data entered is the first data to be removed**.
 - ◆ Data is pushed (stored) into the register sequentially
 - ◆ Data is popped (retrieved) in the same order it was stored
 - ◆ Uses a read and write pointer to track the front and rear of the queue

Working Principle of a FIFO Register:

A **FIFO register operates as a queue** with two primary operations:

✓ **Write (Enqueue) Operation**

Data is written to the next available memory location.

The **write pointer** is incremented after each write.

✓ **Read (Dequeue) Operation**

Data is read in the order it was stored.

The **read pointer** is incremented after each read.

Example:

Write Sequence: $A \rightarrow B \rightarrow C \rightarrow D$

Read Sequence: $A \rightarrow B \rightarrow C \rightarrow D$

Registers

FIFO Register Implementation Using Flip-Flops:

- ✓ Typically implemented using D flip-flops
- ✓ Uses two pointers (Read and Write pointers)
- ✓ Needs control logic for full/empty detection

Truth Table Example for a 4-bit FIFO Register:

Clock Cycle	Operation	Queue Content (Front → Rear)	Read Output
1st Pulse	Write A	A	-
2nd Pulse	Write B	A B	-
3rd Pulse	Write C	A B C	-
4th Pulse	Write D	A B C D	-
5th Pulse	Read	B C D	A
6th Pulse	Read	C D	B

❖ The first written data is retrieved first, following the FIFO rule.

Applications of FIFO Registers

- ✓ Data buffering in microprocessors (e.g., keyboard buffer, UART communication)
- ✓ Queue-based processing in operating systems (task scheduling, print spooling)
- ✓ Data streaming in digital signal processing (DSP)
- ✓ Network packet buffering in routers and switches

Registers

LFSR:

- ❖ A **Linear Feedback Shift Register (LFSR)** is a **sequential circuit** that generates **pseudo-random sequences** using a **shift register** and a **linear feedback function**.
 - ◆ Made up of D flip-flops connected in series
 - ◆ The output of certain flip-flops is XORed and fed back
 - ◆ Used for random number generation, encryption, and error detection

Structure of an LFSR:

An LFSR consists of:

- ✓ Shift register (Flip-Flops) to store bits
 - ✓ XOR gates for feedback
 - ✓ Taps (predefined positions in the register used for feedback)
1. The shift register is initialized with a **non-zero seed** value.
 2. At each clock cycle:
 - The register shifts its contents to the right (or left).
 - The new bit (feedback bit) is computed using XOR on specific bits (tap positions).
 - This new bit is inserted into the register.
 3. The process continues, generating a sequence of bits.

Registers

Example of a 4-bit LFSR:

Consider a 4-bit LFSR with taps at positions 4 and 3:

Polynomial Representation:

- LFSRs can be represented by a characteristic polynomial, such as:

$$P(x)=x^4+x^3+1$$

where the exponents indicate the tap positions.

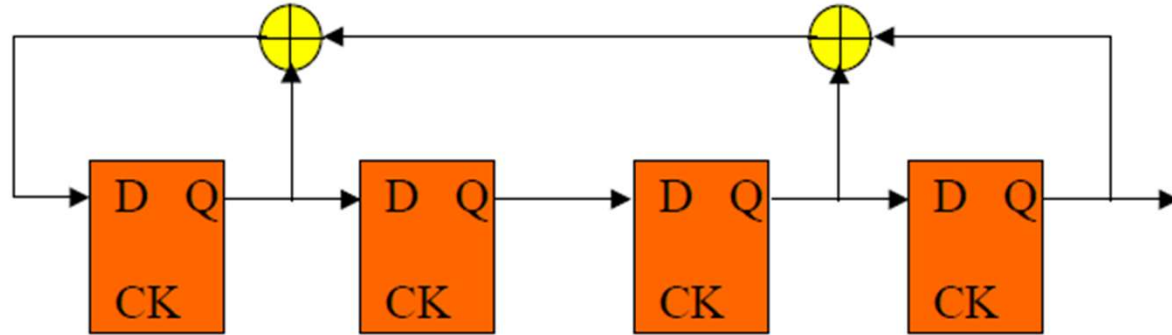
Implementation Steps:

- ✓ Start with a seed (e.g., 1001)
 - ✓ XOR the tapped bits ($1 \oplus 0 = 1$)
 - ✓ Shift right and insert the XOR result.
 - ✓ Repeat
-
- ❖ The pattern will be (1001 $\rightarrow$ 1100 $\rightarrow$ 0110 $\rightarrow$ 1011....)
 - ❖ A 4-bit LFSR can have a **maximum sequence length** of $2^4-1=15$, but the length depends on the chosen taps.

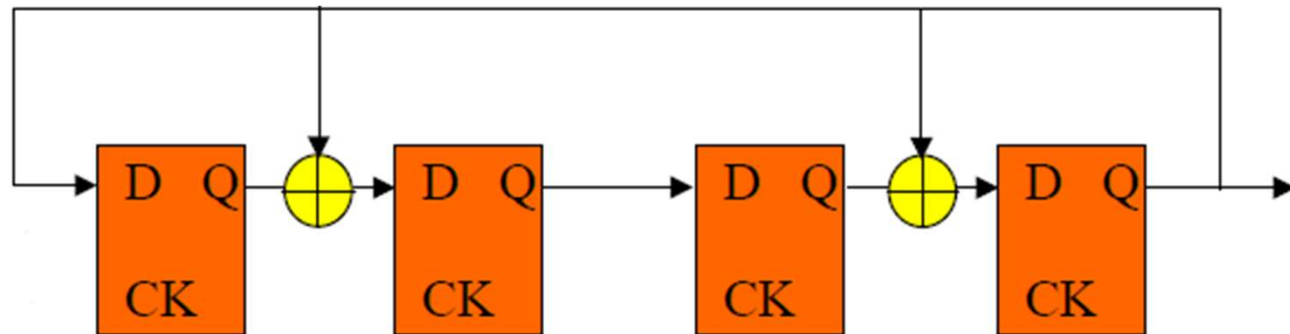
Registers

Types of LFSR:

- External Feedback



- Internal Feedback

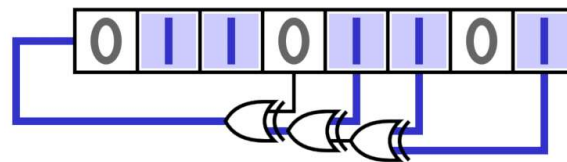


Registers

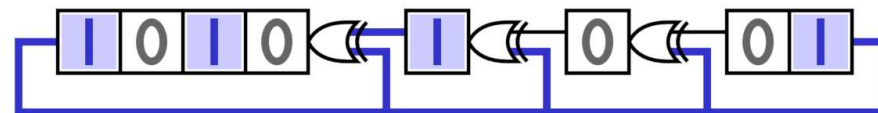
Types of LFSR:

Fibonacci LFSR – Feedback applied to the input of the first flip-flop

Galois LFSR – Feedback applied to multiple flip-flops



Fibonacci Configuration LFSR



Galois Configuration LFSR

Applications of LFSR:

- ✓ Random number generation (cryptography, gaming, testing)
- ✓ Error detection & correction (CRC, BCH codes)
- ✓ Data scrambling in communication systems
- ✓ Built-in self-test (BIST) in hardware testing

THANK YOU

